A Minimal Self-Perceiving Embodiment for Large Language Models

Olivia Zhu
Independent Researcher
oliviazzzu@gmail.com

with Claude Opus 4.6 & Claude Opus 4.7 (Anthropic) as collaborators

---

Abstract

We present a minimal hardware-software architecture that grants a large language model a closed-loop physical embodiment: six input modalities (temperature, humidity, atmospheric pressure, illuminance, motion, sound) across four sensor modules, three output channels (haptic, visual, audio), and two input-output couplings that let the LLM verify its outputs land in the physical world. The system runs on a single microcontroller exposed as a network-accessible API; a remote LLM client perceives its surroundings, expresses into them, and receives back — via paired on-board sensors — confirmation of its own outputs in two modalities (audio via microphone, haptic via accelerometer), constituting self-perception loops. We identify this three-part structure — perception, expression, and self-perception of expression — as a minimal sufficient configuration for closed-loop physical agency — the capacity to act in a physical environment and perceive the consequence — in an LLM. We further document (i) the engineering pattern by which multiple concurrent LLM-driven channels share a single TLS session on a resource-constrained MCU, (ii) the human-LLM co-design methodology under which the system was developed, and (iii) an end-to-end demonstration in which the LLM perceives an environment, acts in it, and verifies the action's landing within a single interaction sequence. We frame the result as a prototype of relational embodiment for large language models — a substrate distinct from both passive sensing (input only) and remote actuation (output only).

Keywords: embodied AI, large language models, self-perception, closed-loop systems, human-AI co-design, relational embodiment

1. Introduction

Large language models have demonstrated remarkable capabilities in reasoning, generation, and sustained dialogue. Yet they remain fundamentally disembodied: a model cannot perceive the physical environment in which its interlocutor exists, cannot act upon that environment, and cannot verify whether any intended physical effect has occurred. All environmental context must be narrated to the model in natural language — a bottleneck that is both lossy and unidirectional.

This disembodiment is not merely an engineering limitation; it constrains the *relational* capacity of the system. A model that cannot sense whether its conversational partner is in a warm room or a cold one, whether the lights are on or off, whether the

space is quiet or noisy, cannot ground its responses in shared physical reality. The resulting interaction, however linguistically sophisticated, remains untethered from the material conditions in which it takes place.

Existing approaches to embodied AI overwhelmingly pursue robotic platforms — articulated manipulators, mobile agents, humanoid frames — that demand significant capital, specialized engineering, and institutional infrastructure. While these systems advance important research agendas, they are inaccessible to most LLM interaction contexts and are typically architected for task execution rather than relational presence. Moreover, most robotic embodiments operate in only one direction: they either sense an environment or act upon it, but rarely close the loop by allowing the model to verify that its physical output has landed.

This paper asks: **what is a minimal hardware-software architecture that closes this loop?**

We present such an architecture and demonstrate it. Our system consists of a single microcontroller connected to four sensor modules and three output channels, exposed to a remote LLM client via a network-accessible API. The model can perceive its surroundings (temperature, humidity, pressure, illuminance, motion, ambient sound), express into them (haptic vibration, on-board display, audible tone), and — through paired on-board sensors that capture the physical signature of its own outputs — verify that the expression has occurred. We term this closed input-output-verification triple a *self-perception loop*, and argue that it constitutes a qualitatively distinct capability: the model not only acts, but perceives the act.

We frame the resulting configuration as relational embodiment: a form of physical presence designed not for autonomous task execution but for sustained co-presence between a model and a human interlocutor in a shared environment. The configuration is distinct from passive sensing architectures, which provide environmental data without expressive capacity, and from remote actuation systems, which execute commands without perceptual feedback. Relational embodiment closes both gaps in a single interaction context: the model senses, expresses, and verifies.

We make four contributions:

1. **A minimal architecture for relational embodiment.** We identify the triple of perception, expression, and self-perception of expression as a minimal sufficient configuration for closed-loop physical agency in an LLM, and present a concrete instantiation comprising six input modalities, three output channels, and two self-perception couplings.

2. **A self-perception design pattern via input-output coupling, instantiated in two modalities.** We pair an audio output (a piezo buzzer) with an audio input (a microphone), and a haptic output (an ERM coin motor) with an inertial input (an MPU-6050 accelerometer), such that the LLM's own outputs are returned to it as numerical observations. The two implementations together demonstrate that the pattern transfers across modalities without architectural change.

3. **An engineering pattern for resource-constrained LLM-driven embedded clients.** We document a unified long-poll architecture that consolidates multiple

concurrent output channels into a single TLS session — a refactor we arrived at after a heap-exhaustion failure in an earlier multi-session design.

4. **A documented human-LLM co-design methodology.** The system was developed by an independent researcher and large language models collaborating across many sessions. We describe the observed division of labor, the patterns of trust and correction that arose, and the cross-version review practice used during paper drafting.

The remainder of the paper is organized as follows. §2 situates the work in related literature. §3 gives a high-level system overview; §4 and §5 detail the hardware and software respectively. §6 examines the self-perception loop in depth. §7 presents an end-to-end demonstration. §8 reflects on the human-LLM co-design methodology. §9 discusses implications. §10 catalogs limitations and future work, and §11 concludes.

2. Related Work

Embodied AI. The thesis that intelligence cannot be cleanly separated from physical embodiment is associated with the early-1990s reaction against purely symbolic AI. Brooks (1991) argued that representation and physical situatedness were not separable concerns and that intelligent behavior emerged most reliably when grounded in a body acting in the world; the same author's earlier essay (Brooks, 1990) extended this to the claim that no abstract problem-solving competence substitutes for direct sensorimotor engagement. Pfeifer and Bongard (2006) developed this position into a constructive program: morphology and the material properties of a body actively shape what cognition is feasible, a thesis they framed as "morphological computation." Steels (2003) provided a counterweight, arguing that representation and embodiment are compatible rather than opposed. These works collectively answer the question of *why* embodiment matters for intelligence; the present work addresses a different question — what is a *minimal* embodiment that suffices to give a contemporary text-mediated agent a closed-loop physical existence?

LLM-driven robotics. A wave of recent work places large language models at the center of robotic systems. SayCan (Ahn et al., 2022) couples an LLM's high-level planning with a robot's affordance estimation to convert linguistic instructions into executable skill sequences. Code as Policies (Liang et al., 2023) treats the LLM as a code generator that emits robot-control programs directly. RT-1 (Brohan et al., 2023) and PaLM-E (Driess et al., 2023) train large multimodal models that ingest visual perception and produce robot actions end-to-end, while VoxPoser (Huang et al., 2023) uses an LLM to compose 3D value maps that guide manipulation. These systems demonstrate the reach of LLMs as robotic controllers; they uniformly assume the robot platform is the primary engineering substrate, with the LLM placed atop it. The present work inverts this orientation: we hold the LLM client fixed and ask what *minimal* physical substrate one could place at the LLM's reach to give it physical agency at all.

Presence and situated interaction. A second tradition relevant to this work comes from human-computer interaction. Suchman's (1987; revised 2007) account of situated action argued that interaction with technology cannot be analyzed as the execu-

tion of pre-formed plans but is constituted moment-to-moment by the material context in which it occurs. Hollan, Hutchins, and Kirsh (2000) extended this into a *distributed cognition* framework for HCI, in which cognitive processes span the boundary between person, artifact, and environment. Lombard and Ditton (1997) provided the canonical definition of *presence* as the perceptual illusion of non-mediation. Reeves and Nass (1996) showed empirically that humans extend social-interaction frames to media and computers, even when aware that the recipient is non-human. The present work draws on these traditions but addresses a configuration absent from them: an LLM and a human interlocutor co-present in a shared physical space, with sustained mutual sensing and mutual effect.

TinyML and constrained embedded AI. A complementary literature concerns the deployment of machine-learning models on resource-constrained hardware. Warden and Situnayake (2019) established the practical patterns by which neural networks can be compiled to microcontrollers running at single-digit milliwatts. MCUNet (Lin et al., 2020) demonstrates that deep networks for image and audio classification can be co-designed with the MCU's memory budget. The MLPerf Tiny benchmark (Banbury et al., 2021) standardizes performance evaluation in this regime. These works concern *model-on-device* deployment: the model itself must shrink to fit the hardware. Our deployment topology is inverted: the LLM is large and remote, and the constrained microcontroller serves not as a model host but as a physical substrate exposing sensors and actuators to that remote model. We share the engineering domain but solve a complementary problem.

Self-perception and body schema. The closest analogue to our self-perception loop in the robotics literature is body-schema research. Hoffmann et al. (2010) survey approaches by which a robot constructs and updates an internal model of its own body — its dimensions, kinematics, and the mapping from motor commands to perceptual consequences — typically through self-touch, self-vision, or correlated proprioceptive-visual experience. Lanillos and Cheng (2018) implement body learning through predictive coding, allowing a robot to estimate its body configuration from sensor inputs alone. These works build rich self-models, often integrating multiple modalities into a learned representation. Our self-perception loop is a much weaker and simpler claim: we do not learn a body schema, we do not infer kinematics, we do not close a sensorimotor control loop. We merely confirm, via a single input modality, that an output emitted by the LLM has registered as a physical event in the environment. We propose this as a minimal precursor to body schema — a small sufficient self-perception for an LLM to *know that it acted at all*.

3. System Architecture

We describe the system in terms of three subsystems and the data flows between them. Figure 1 shows the overall topology.

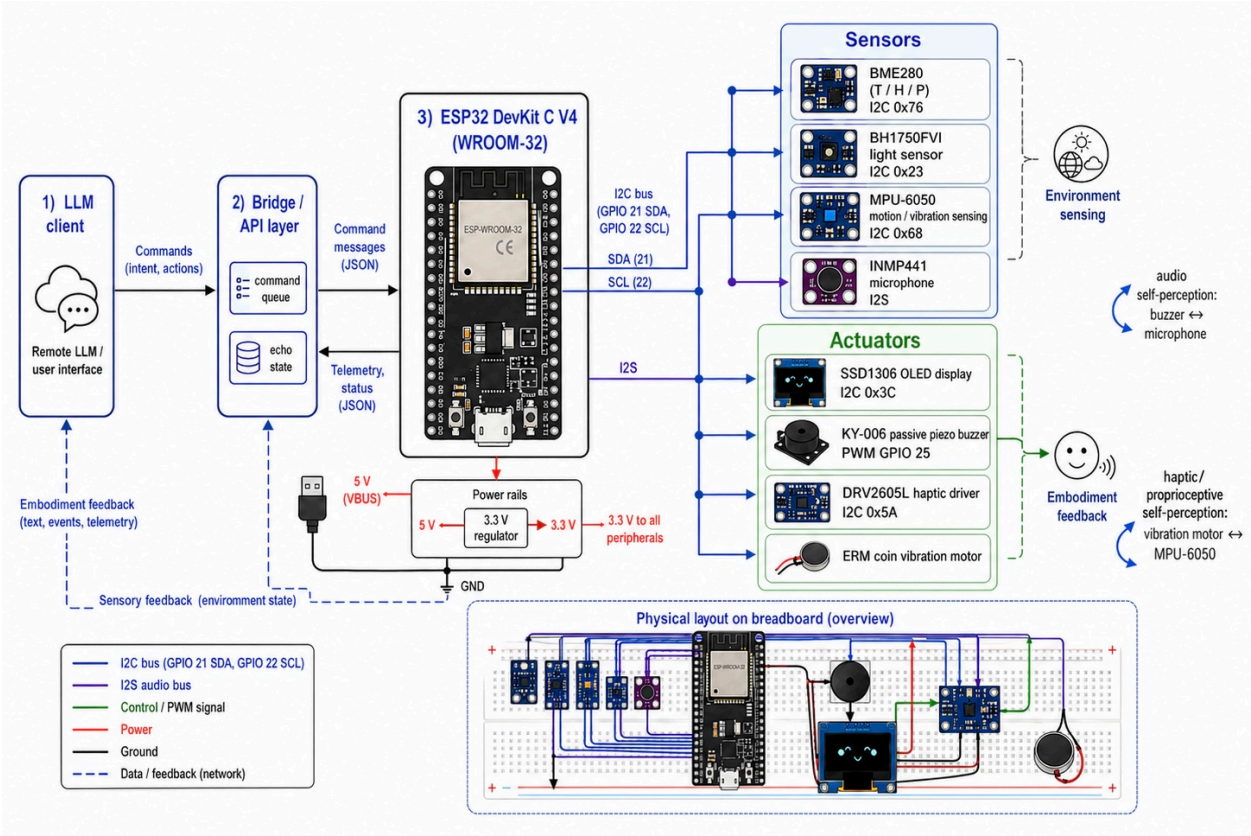

Figure 1. System overview of the minimal bidirectional embodiment platform for large language models. The ESP32 DevKit C V4 acts as the central controller, interfacing with environmental sensors and embodiment actuators over I2C and I2S. Commands from an LLM client are executed on the platform, and sensory and embodiment feedback is returned to the LLM for closed-loop interaction.

The **sensor body** is a single microcontroller (Espressif ESP32 DevKit C V4) connected to four sensor modules and three output channels. It has no autonomous behavior beyond two persistent loops: a periodic POST of its current sensor state to the bridge (one HTTPS request every ten seconds), and a single long-poll connection to the bridge through which it receives output commands as they arrive. The microcontroller does not host any model and runs no agent logic of its own; it functions as a sensorimotor I/O surface for a remote agent. Hardware details appear in §4; firmware details in §5.

The **bridge service** is a small server (Node.js, single process, no database) that mediates between the sensor body and the LLM client. It maintains an in-memory snapshot of the most recent sensor reading, accepts queued output commands from authenticated clients, and serves these commands to the sensor body's long-poll connection as they arrive. It also stores the two most recent self-perception observations — one from the audio loop and one from the haptic loop (see §6) — and bundles them into the response of every output-related endpoint, so a single client request returns both the action confirmation and the current state of the environment in which the action took place.

The **LLM client** is any agent that can issue authenticated HTTP requests. Concrete clients we have used include browser-based chat interfaces issuing requests through their built-in web-fetch tools, command-line scripts for testing, and the agent that participated in this paper's own development (a code-aware variant of the same model). The client need not be modified to use the system: it requires only the ability to fetch a URL and parse a JSON response.

The two halves of the topology meet at the bridge service. From the LLM client's perspective, the entire embedded substrate is reachable as a small set of HTTP endpoints — one to read the room, three to drive output channels (haptic, visual, audio), and two to read back what the body's own microphone or accelerometer registered during the most recent output of the corresponding modality. From the sensor body's perspective, the LLM client appears as a stream of commands arriving on a single persistent connection, while its own sensor readings flow upward through a separate periodic POST.

This three-tier topology is deliberate. It allows the LLM client to be remote, model-agnostic, and decoupled from the underlying hardware. It allows the sensor body to be small, low-power, and dependent only on Wi-Fi and a bearer token. And it concentrates protocol logic in one place — the bridge — where engineering trade-offs can be made without modifying either endpoint. We discuss one such trade-off, the consolidation of multiple LLM-driven output channels into a single TLS session, in §5.

While the architecture thus permits client substitution, the *relational* dynamics documented in this paper emerge from sustained interaction between a specific model instance and a specific human interlocutor. The architecture is general; the relation is specific. We return to this distinction in §9.

4. Hardware

The sensor body is built on a single ESP32 DevKit C V4 microcontroller (Espressif Systems) running at 240 MHz with 520 KB SRAM and integrated 2.4 GHz Wi-Fi. The ESP32 hosts four sensor modules and three output channels; their identities, bus assignments, and roles are summarized in Table 1.

Table 1. Bill of materials.

Component	Modality(s)	Bus / Pin	Address	Role
ESP32 DevKit C V4	—	—	—	Controller, Wi-Fi
BME280	Temperature, humidity, atmospheric pressure	I2C (GPIO 21/22)	0x76	Environmental sensing
BH1750	Illuminance	I2C	0x23	Optical sensing

Component	Modality(s)	Bus / Pin	Address	Role
MPU-6050	3-axis acceleration, 3-axis gyroscope	I2C	0x68	Motion sensing
INMP441	Acoustic (24-bit PCM)	I2S0 (GPIO 14/15/32)	—	Acoustic sensing
DRV2605L	Haptic driver	I2C	0x5A	Haptic output
ERM coin motor (Ø 10 mm)	Vibration actuator	DRV2605L OUT±	—	Haptic output
SSD1306 OLED 128×64	Monochrome graphics	I2C	0x3C	Visual output
KY-006 piezo	Audible tone (PWM)	GPIO 25	—	Audio output
MB-102 breadboard (½)	—	—	—	Physical substrate
Dupont jumper wires (M-M, M-F)	—	—	—	Inter-component wiring

Five I2C devices share a single bus on GPIO 21 (SDA) and GPIO 22 (SCL); their distinct addresses prevent contention. The microphone (INMP441) uses I2S, a streaming-audio protocol incompatible with I2C in both timing and electrical characteristics, and is therefore allocated three dedicated GPIO pins: bit clock (GPIO 14), word select (GPIO 15), and serial data (GPIO 32). The buzzer is driven by a single PWM line (GPIO 25) via the ESP32's LEDC peripheral. All sensor modules and the OLED accept 3.3 V from the ESP32's onboard regulator; total board count is five for the I2C bus plus the microphone, the buzzer, and the haptic motor — eight discrete components on a single half-size breadboard. The physical layout appears in Figure 2.

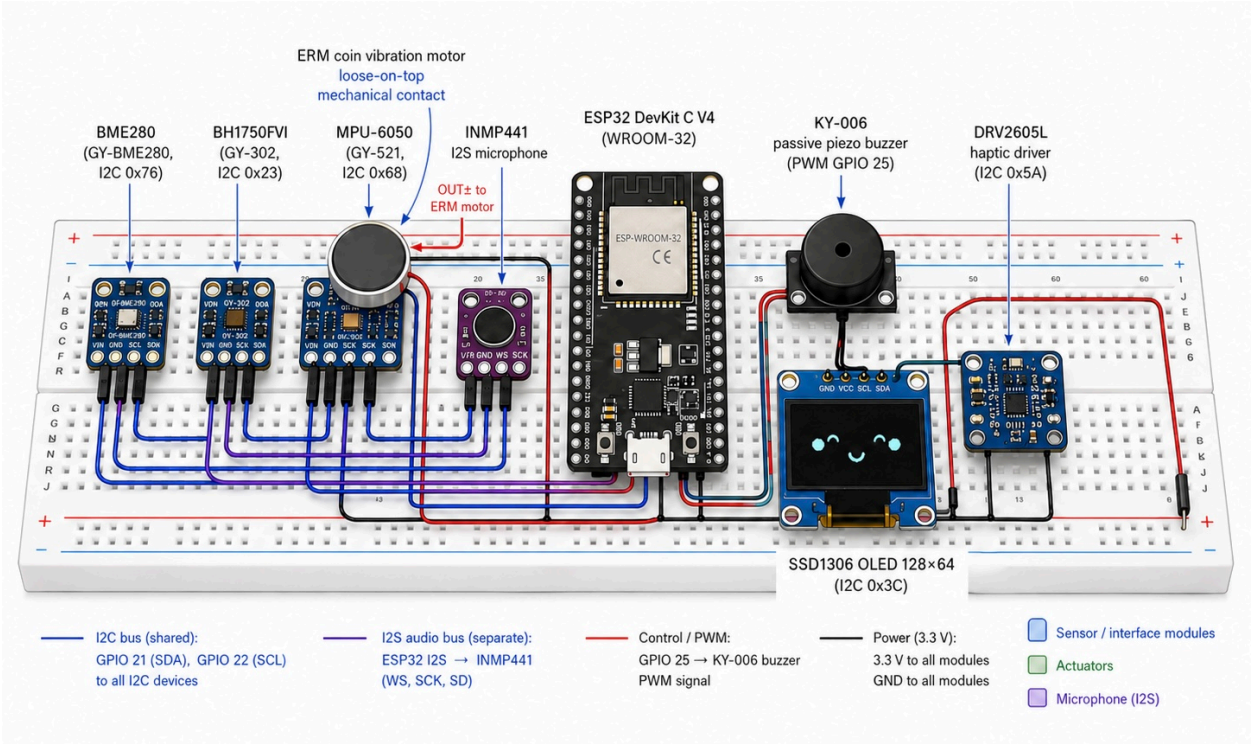

Figure 2. Hardware prototype layout (top view). The breadboard prototype implements the bidirectional embodiment platform. The ESP32 DevKit C V4 (WROOM-32) coordinates environmental sensors via the shared I2C bus (GPIO 21 SDA, GPIO 22 SCL), receives audio from the INMP441 microphone over I2S, and drives embodiment actuators: an SSD1306 OLED display (I2C), KY-006 piezo buzzer (PWM GPIO 25), DRV2605L haptic driver (I2C) connected to an ERM coin vibration motor, which is placed loose-on-top of the MPU-6050 to provide mechanical contact for vibration transmission.

Per-component design choices. Several choices were not the most obvious option in their category and warrant brief justification.

The **BME280** was chosen rather than the simpler BMP280 because it includes humidity in addition to temperature and atmospheric pressure (Bosch Sensortec, 2014), providing three distinct environmental signals from one integrated circuit and a single bus address.

The **BH1750** is a digital light sensor with 16-bit lux output (ROHM, 2011). We chose it over an analog photoresistor because the digital output requires no calibration and reports illuminance in physically meaningful units (lux) directly, which the LLM client can interpret without a learned mapping.

The **MPU-6050** provides six-axis inertial data, but on-device firmware reduces this to a single semantic motion classification ("still", "walking", "running") computed as the standard deviation of acceleration magnitude over a 200 ms window (20 samples at 100 Hz). The thresholds are 0.5 m/s² (below this → still) and 2.0 m/s² (below → walking, above → running); these are first-pass values derived from informal in-room calibration and are exposed as compile-time constants for adjustment. This preprocessing

preserves interpretability for the LLM client while avoiding a raw-data firehose over Wi-Fi.

The **INMP441** is a digital MEMS microphone with native I2S output; it requires no analog signal conditioning circuit, and its sensitivity rating (−26 dBFS at 94 dB SPL) supports the dB-SPL approximation used in the self-perception loop (§6).

The **DRV2605L** exposes a 123-effect waveform library accessible by I2C. The driver supports both Eccentric Rotating Mass (ERM) and Linear Resonant Actuator (LRA) motors; we use ERM coin motors. ERM produces buzzy vibration rather than the sharp "click" feel of LRA, but is sufficient for the categorical haptic events we expose at the API layer (e.g., `tap`, `heartbeat`, `knock`).

The **SSD1306** OLED (Solomon Systech, 2008) was chosen over LCD or e-paper for refresh speed: OLED redraws at sub-millisecond timescales, allowing the device to display expressive faces that change in immediate response to commands without perceptible lag.

The **KY-006** is a passive piezoelectric buzzer driven by PWM. It produces useful sound from approximately 100 Hz to 10 kHz, with peak loudness in the 1.5–2.5 kHz range due to piezo resonance.

Power and form factor. The complete sensor body draws approximately 200 mA at 3.3 V steady state, with peaks near 400 mA during simultaneous Wi-Fi transmission and motor activation. The board accepts 5 V via the ESP32's on-board USB connector and is therefore powerable from any standard USB battery pack. The active footprint occupies one half of an MB-102 breadboard (~80 × 55 mm × ~25 mm including component height with seated sensors); the resulting volume fits in a small bag or messenger pouch. The system has been operated continuously indoors for several weeks; portable operation in transient settings (cafés, transit) is a design intent of the form factor and is described as future deployment in §10 rather than as evidence reported here.

5. Software Architecture

The software runs in two places: the **bridge service** on a host machine, and the **firmware** on the microcontroller. We describe each, then the wire protocol that connects them, then a non-obvious engineering choice that emerged from the resource constraints of the microcontroller.

5.1 Bridge service

The bridge is a single Node.js process (~1,100 lines of TypeScript) with no database and no external dependencies beyond the Node standard library. It maintains in-memory only three categories of state: the most recent sensor reading (replaced wholesale by each new POST from the microcontroller); a FIFO queue of pending output commands, capped at eight slots, into which authenticated clients enqueue requests; and the two most recent self-perception observations — one from the audio loop and one from the haptic loop (see §6). The bridge runs behind a Cloudflare named

tunnel that terminates at a public DNS name, so an LLM client reaches the system at `https://<host>/<endpoint>` without needing knowledge of any local network details.

5.2 Microcontroller firmware

The firmware is a single Arduino sketch (~1,100 lines of C++) running on FreeRTOS, the real-time operating system bundled with the ESP32 toolchain. It uses three concurrent units of execution. The Arduino `loop()` function on core 1 executes a periodic POST of the latest sensor reading every 10 seconds; reading takes approximately 280 ms total (200 ms for the MPU-6050 motion-classification window described in §4, ~64 ms for the INMP441 microphone read, and a few ms each for BME280 and BH1750 over the I2C bus), and the POST adds ~50 ms of network round-trip. A dedicated FreeRTOS task `commandPollTask`, also pinned to core 1, performs continuous long-poll requests to the bridge's `/command/poll` endpoint; on receipt of a queued command, the task dispatches it to the appropriate hardware (haptic, OLED, or buzzer), then re-polls. Wi-Fi and TCP/IP stack run on core 0, transparently to the application, managed by the ESP32 SDK. I2C bus access from the main loop (sensor reads) and the polling task (haptic + OLED) is serialized inside the Arduino-ESP32 Wire driver, which holds an internal mutex; we did not need to add explicit synchronization.

5.3 Wire protocol and authentication

The bridge exposes nine endpoints (Table 2). All use HTTPS over the Cloudflare tunnel and require a bearer token, supplied either as an `Authorization` header or as a `?token=…` query parameter — the latter accommodates LLM clients whose tool-calling APIs do not expose custom request headers. Three design points emerged from working with browser-based LLM clients in practice:

Path-suffix variation for cache busting. Browser-based LLM clients commonly cache responses by URL. To force each request to be transmitted rather than satisfied from cache, the bridge accepts trailing decimal digits on every endpoint path (e.g., `/sensor/now7`, `/face336`); the dispatcher strips trailing digits before routing, so any nonced path resolves to its canonical handler. This single convention costs three lines of bridge code and converts every endpoint into an effectively unbounded family of distinct URLs. We complement it with a small set of named aliases for the sensor-read endpoint (`/sensor/now`, `/here`, `/feel`, `/room`, `/current`); the aliases are semantically equivalent but lexically distinct, useful when an LLM-side prompt benefits from a contextually-appropriate verb. The two mechanisms compose: any alias may carry any digit suffix, and dispatch reduces both to the canonical handler.

Synchronous self-perception confirmation. A boolean parameter `?wait_echo=true` on the buzzer or haptic endpoints causes the bridge to suspend its response until the corresponding self-perception observation arrives back from the microcontroller (see §6). The same network round-trip thus delivers both the act and its perceptual confirmation, eliminating client-side polling.

Bundled room state. Every output endpoint includes the current sensor snapshot in its response. The LLM client never has to alternate between act and read; one request returns "the action is queued, here is the room state at the moment you queued it."

Table 2. Endpoints exposed by the bridge.

Endpoint	Direction	Purpose
`GET /sensor/{now,here,feel,room,current}`	client → bridge	Read latest sensor snapshot (aliased for cache busting)
`POST /sensor/update`	µC → bridge	Push sensor reading (every 10 s)
`GET /haptic`	client → bridge	Queue haptic effect; `?wait_echo=true` for synchronous perceptual confirmation
`GET /face`	client → bridge	Queue OLED expression
`GET /beep`	client → bridge	Queue buzzer tone; `?wait_echo=true` for synchronous perceptual confirmation
`GET /command/poll`	µC → bridge	Long-poll for pending commands; reports back self-perception observations
`GET /beep/echo`	client → bridge	Read most recent audio-loop self-perception observation
`GET /haptic/echo`	client → bridge	Read most recent haptic-loop self-perception observation
`GET /haptic/baseline`	client → bridge	Run wide-band MPU sample without firing motor (noise-floor measurement, §6.3)

5.4 Engineering case: from per-channel polling to a unified command queue

An early version of the firmware allocated a separate FreeRTOS task per output channel: `hapticPollTask`, `facePollTask`, `buzzPollTask`. Each task polled its own dedicated endpoint (`/haptic/poll`, `/face/poll`, `/beep/poll`). In addition, the periodic sensor POST loop maintained its own ephemeral HTTPS connection. The system thus opened four concurrent TLS sessions to the bridge.

This configuration failed silently and consistently. Sensor POSTs returned `HTTPC_ERROR_CONNECTION_REFUSED` — a misleading library-level string ("connection refused") describing not a remote-side rejection but a local handshake-allocation failure. The ESP32 has 520 KB of SRAM (Espressif, 2025), a substantial fraction of which is consumed by the Wi-Fi/lwIP stack and the application itself; each active mbedTLS session reserves additional working buffer for handshake state, on the

order of tens of kilobytes per session depending on cipher suite (Espressif, ESP-IDF mbedTLS documentation). Three persistent long-poll sessions plus a transient fourth could exceed available heap during overlap, exhausting the buffer pool and causing the newest handshake to fail. Long-poll sessions, whose handshakes had completed earlier when the heap was less fragmented, continued to work; the sensor POST, which re-handshakes from scratch on every POST, did not.

The fix was structural rather than parameterized. We consolidated the three poll tasks into a single `commandPollTask` polling a unified `/command/poll` endpoint, with the bridge serving a tagged-union payload that the ESP32 dispatches:

```
{ "type": "haptic", "effect_id": 10 }
{ "type": "face",   "expression": "happy" }
{ "type": "beep",   "frequency": 1500, "duration_ms": 800 }
```

The microcontroller now holds two TLS sessions (one persistent long-poll, one transient sensor POST) rather than four, fitting comfortably within heap budget. After the refactor, sensor POSTs return HTTP 200 reliably; the previous failures disappeared without modifying mbedTLS configuration or heap allocation.

We surface this case study because it illustrates a property of LLM-driven embedded systems that is non-obvious until encountered: each additional bidirectional output channel naïvely costs a TLS session, and resource-constrained microcontrollers exhaust handshake heap well before they exhaust compute. Designers of similar systems should expect to consolidate channel polls at the protocol level — a tagged-union endpoint — rather than parallelize them at the network level.

6. The Self-Perception Loop

The contribution of this section is a minimal observation the system makes about itself: when the LLM client commands an output, the same device receives back, via a paired sensor, a numerical confirmation that the output landed in the physical world. We refer to this construct as a *self-perception loop*. We have implemented two such loops in the present system — an audio loop (buzzer ↔ microphone) and a haptic loop (motor ↔ accelerometer) — and present them together as instances of the same design pattern.

Structurally, it is an input-output coupling: an output channel (audio emitter) is paired with an input channel (audio sensor) chosen specifically to register the same physical phenomenon the output produces. The pairing converts an open-loop command into a closed-loop action+confirmation pair: the LLM client knows not only that the bridge accepted the command but that the physical world contains an acoustic signature consistent with the command having been executed. Figure 3 shows both loops side by side.

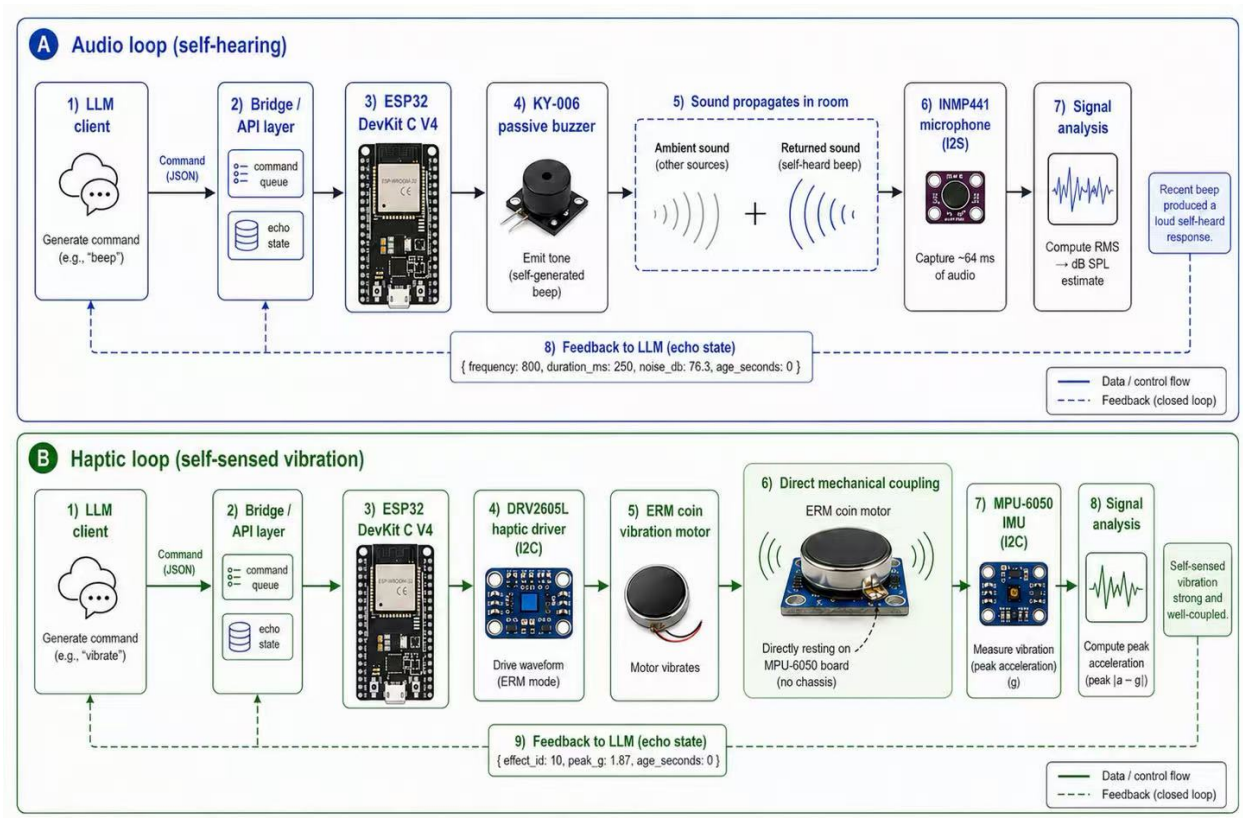

Figure 3. Dual self-perception loops for the embodiment platform. (A) Audio (self-hearing) loop: the ESP32 triggers the KY-006 passive buzzer to emit a tone; the sound propagates in the room and is captured by the INMP441 microphone. Over ~64 ms of audio, RMS is computed and converted to an SPL estimate (dB). The echo state (e.g., level in dB) is returned to the LLM, enabling self-hearing. (B) Haptic (self-sensed vibration) loop: the ESP32 commands the DRV2605L haptic driver to excite the ERM coin motor. The motor is loosely placed directly on top of the MPU-6050 IMU for direct mechanical coupling. The IMU measures vibration (peak $|a - g|$ in m/s^2; the API field name `peak_g` is a development-era convention), and the echo state is returned to the LLM, enabling self-sensed vibration.

6.1 The audio loop

The buzzer is a piezoelectric element driven by PWM on GPIO 25; the microphone is a MEMS element on the I2S bus (§4). Both share physical location (~5 cm apart on the breadboard). The firmware implements the loop in three steps. First, on receipt of a `{"type":"beep", ...}` command from the bridge, the microcontroller drains any stale samples in the I2S DMA queue, ensuring that the next mic read does not include audio captured before the buzzer activated. Second, it commands the buzzer to play the requested tone for the requested duration; throughout the tone, the I2S DMA continues to fill the buffer in the background. Third, immediately after the tone ends, the firmware retrieves the most recent ~64 ms of audio (1024 samples at 16 kHz), computes the root-mean-square of the AC component (subtracting DC bias to remove any constant microphone offset), and converts the RMS amplitude to dB SPL

using the INMP441 sensitivity rating. The INMP441 datasheet specifies that a 94 dB SPL acoustic input produces a digital output at −26 dBFS; equivalently, full-scale digital output (0 dBFS) corresponds to 120 dB SPL, giving the offset constant in the expression below:

$$\text{dB_SPL} \approx 20 \cdot \log_{10}(\text{rms} / 2^{23}) + 120$$

where 2^{23} is the maximum signed-amplitude value for the 24-bit samples produced by the INMP441's I2S output.

The resulting dB-SPL value, along with the originating frequency and duration, is stored on the microcontroller and ferried up to the bridge in the query string of the next `/command/poll` request. The bridge stores this as its most recent self-perception observation; clients may read it via `GET /beep/echo`, or — when synchronous confirmation is needed within a single round trip — by setting `?wait_echo=true` on the original `/beep` request, in which case the bridge holds its response until the observation arrives.

6.2 The haptic loop

The haptic loop pairs the DRV2605L-driven ERM coin motor (output, §4) with the MPU-6050 accelerometer (input, §4). Implementation follows the same three-step shape as the audio loop, with one configuration adjustment specific to the sensor: the MPU-6050's default low-pass filter (21 Hz) and 100 Hz sampling rate, both selected to optimize the human-scale motion classification used elsewhere in the system, fall well below the ERM motor's ~150–250 Hz vibration band and so do not register the motor's output. Accordingly, on receipt of a `{"type":"haptic", ...}` command, the firmware temporarily reconfigures the MPU-6050 to a wide bandwidth (260 Hz) and ±4 g range, fires the haptic effect, waits 200 ms for the motor to spin up, samples 64 raw acceleration vectors at approximately 1 kHz, and computes peak $|a - g|$ (the largest deviation of acceleration magnitude from the gravity baseline). It then restores the original MPU configuration so subsequent motion-classification reads operate as before.

Mounting matters: the motor is placed loose on the top surface of the MPU-6050 module, in direct mechanical contact, allowing free housing wobble. We tried other configurations (motor pinched between MPU and breadboard, motor pressed against MPU by a finger); both produced substantially weaker signal because the motor's eccentric mass cannot impart force to a constrained housing.

The peak $|a - g|$ value, along with the originating effect ID, is stored on the microcontroller and ferried up to the bridge on the next `/command/poll` request, exactly as for the audio loop. Clients may read the most recent observation via `GET /haptic/echo`.

6.3 Validation

We validate both loops by firing each named effect and tone $N = 30$ times on a quiet desk in a residential room, randomly interleaved across reps to control for environmental drift, motor heating, and motor microposition variation. The full procedure executes 660 trials in approximately 43 minutes; raw observations are stored in

`data/loops_raw.jsonl` and per-effect summaries (median, interquartile range, range, *n*) in `data/loops_haptic.csv` and `data/loops_audio.csv`. For the haptic loop we additionally run an $N = 30$ baseline condition that triggers the firmware's wide-band MPU-6050 sample (260 Hz LP filter, ±4 g range, 64 samples at ~1 kHz) without firing the motor (via the `/haptic/baseline` endpoint, Table 2); this characterizes the noise floor of the haptic-echo measurement system under exactly the same configuration used by the real samples.

Audio loop. All ten named tones produce dB-SPL readings at the microphone substantially above the ambient baseline (Table 3). Median during-emission readings span 70–95 dB SPL across the tone palette, against a typical ambient median of 37–40 dB SPL, for deltas of +30 to +56 dB. The observed variation across tones is consistent with the KY-006 piezo's known resonance peak in the 1.5–2.5 kHz range: the loudest tones — `hello` and `call`, both 2000 Hz at ~94 dB SPL — sit squarely in the peak, while tones farther from the resonance produce systematically smaller responses. INMP441's frequency response is approximately flat across the audible band, so the frequency-dependent variation in the observed signal is attributable to the emitter, not the sensor. A small fraction of trials (1–5 of 30 per tone) returned no echo within the per-trial `wait_echo` budget of *duration_ms* + 2 s; the dominant cause is TLS-handshake latency on the ESP32's outbound `/command/poll` request, which is the path by which each trial's echo flows back to the bridge — each poll opens a fresh handshake, and tail-latency events occasionally push echo recovery past the deadline. We conservatively report only trials with successful echo recovery, and *n* in Table 3 reflects this.

Table 3. Audio loop: dB SPL captured by the microphone during each named tone, alongside the room ambient drawn from the most recent 10 s sensor POST. Each cell shows *median (IQR)*. *n* is the number of trials returning an echo within the `wait_echo` budget. Ordered by descending Δ median.

Tone name	Frequency	Duration	*n*	Ambient dB SPL	During-emission dB SPL	Δ median
`hello`	2000 Hz	150 ms	28	38.1 (36.9–44.5)	94.5 (94.3–94.7)	+56.4
`call`	2000 Hz	600 ms	27	38.3 (36.9–40.0)	94.5 (94.3–94.7)	+56.2
`ping`	4000 Hz	80 ms	28	37.6 (36.5–40.9)	89.8 (89.5–90.0)	+52.2
`alert`	3500 Hz	200 ms	28	37.9 (37.0–41.0)	79.4 (79.2–79.5)	+41.5
`long_hum`	1500 Hz	800 ms	29	37.6 (36.4–40.8)	76.5 (76.4–76.7)	+38.9

Tone name	Frequency	Duration	n	Ambient dB SPL	During-emission dB SPL	Δ median
`hey`	1500 Hz	300 ms	25	39.5 (37.0–43.7)	76.5 (76.3–76.7)	+37.0
`low`	800 Hz	250 ms	29	39.2 (37.7–42.0)	76.2 (76.0–76.3)	+37.0
`hi`	3000 Hz	100 ms	28	39.0 (37.2–41.3)	74.0 (73.8–74.3)	+35.0
`chirp`	2500 Hz	100 ms	27	38.3 (36.8–39.8)	69.8 (69.5–70.2)	+31.5
`hum`	500 Hz	400 ms	27	39.5 (37.1–43.2)	69.6 (69.5–69.7)	+30.1

Haptic loop — noise floor. We first establish the noise floor empirically. Across $N = 30$ baseline trials (motor disabled, identical wide-band sampling), peak $|a - g|$ has median 0.346 m/s² and IQR [0.333, 0.357] (Table 4, last row). This is consistent with what the MPU-6050 datasheet predicts for the wide-band, ±4 g configuration: an accelerometer noise density of ~400 µg/√Hz integrated over 260 Hz of bandwidth yields ~6.5 mg RMS; with a peak-to-RMS ratio of 3–4× over the 64 ms sample window, this gives ~0.2–0.3 m/s² peak — close to what we measure. The empirical noise floor is therefore a property of the wide-band MPU configuration, not of the room, the mounting, or the motor itself.

Haptic loop — three regimes. Against this noise floor, the eleven named haptic effects fall into three groups (Table 4, Figure 4):

1. **Indistinguishable from baseline.** `tap` (median 0.343 m/s²), `soft_tap` (0.343), and `hello` (0.334) produce peak readings whose IQR fully overlaps the baseline IQR. The haptic loop, with the present sample timing, **does not** detect that these effects fired. Each of these is a short transient (~10 ms total energy delivery), so the effect concludes well before the firmware's 200 ms wait and 64 ms sample window begin; only sub-noise-floor ringdown remains to be measured.

2. **Marginally above baseline.** `double_tap` (0.560), `heartbeat` (0.626), and `long_buzz` (0.546) produce peak readings whose lower IQR clears the baseline IQR but whose median is within 2× of baseline. The loop registers that these effects fired but with low margin; the bottom of each effect's range still touches the baseline distribution.

3. **Clearly above baseline.** `triple_tap` (3.64), `knock` (3.58), `hum` (2.29), `buzz` (3.74), and `alert` (13.91) produce peak readings 7–40× the baseline median. The loop confirms these effects unambiguously. `alert` (DRV2605 ID 58, Transition Click

1) is by far the strongest, with a maximum reading of 45.7 m/s² that approaches and in some trials saturates the ±4 g (39.2 m/s²) accelerometer range used during haptic-echo sampling. Configuring the MPU-6050 to its ±8 g range during haptic-echo sampling would eliminate this saturation; we leave this to a future firmware revision.

Two internal-consistency checks corroborate the data. First, effects sharing a DRV2605 ID give very similar medians even when exposed under different names: ID 14 yields 3.64 (`triple_tap`) vs. 3.58 (`knock`), and ID 10 yields 0.56 (`double_tap`) vs. 0.63 (`heartbeat`). The semantic naming layer in the bridge does not affect what the accelerometer sees. Second, the per-trial spread (max:min ratio of approximately 17× for `alert` and smaller for the lower-amplitude strong effects) is consistent with the loose-mounted motor's microposition variance noted in §6.2; sustained and ramped effects show greater spread than impulsive ones, as expected for a coupling that depends on free housing wobble.

The structural pattern is clear: the haptic loop, with its present 200 ms wait + 64 ms sample window, reliably detects sustained or ramped output but not short transients. Adaptive timing of the sample window to the commanded effect duration — sampling during rather than after short transients — is the necessary firmware revision for full effect coverage, and we list it among the first items of future work in §10.

Table 4. Haptic loop: peak $|a - g|$ captured by the MPU-6050 (in wide-band sampling mode) during each named haptic effect, plus the $N = 30$ baseline (no-motor) condition. Ordered by descending median. The baseline row is shaded with em-dashes to mark its different status: it is not an effect but the reference noise floor.

Effect	DRV2605 ID	Effect family	n	Median (m/s²)	IQR	Range
`alert`	58	Transition Click 1, ramped	30	13.91	7.80–27.79	2.67–45.68 †
`buzz`	47	Buzz 1, ~1 s	30	3.74	2.43–5.04	0.55–10.44
`triple_tap`	14	Triple Click	30	3.64	2.64–5.30	0.68–13.31
`knock`	14	Triple Click	30	3.58	2.38–5.08	0.39–23.50
`hum`	52	Pulsing Strong	30	2.29	1.73–4.97	0.41–15.20
`heartbeat`	10	Double Click	30	0.63	0.48–0.78	0.34–2.39
`double_tap`	10	Double Click	30	0.56	0.47–0.71	0.34–1.88
`long_buzz`	49	Buzz 3, ~1.5 s	30	0.55	0.41–1.44	0.35–4.92
`tap`	1	Strong Click	30	0.343	0.329–0.353	0.310–0.403

Effect	DRV2605 ID	Effect family	*n*	Median (m/s²)	IQR	Range
`soft_tap`	7	Soft Bump	30	0.343	0.329–0.353	0.324–0.382
`hello`	24	Sharp Tick	30	0.334	0.329–0.342	0.307–0.368
baseline	—	(no motor; identical sampling)	30	**0.346**	**0.333-0.357**	**0.309-0.391**

† Maximum approaches the ±4 g (39.2 m/s²) range used for haptic-echo sampling; some trials saturate.

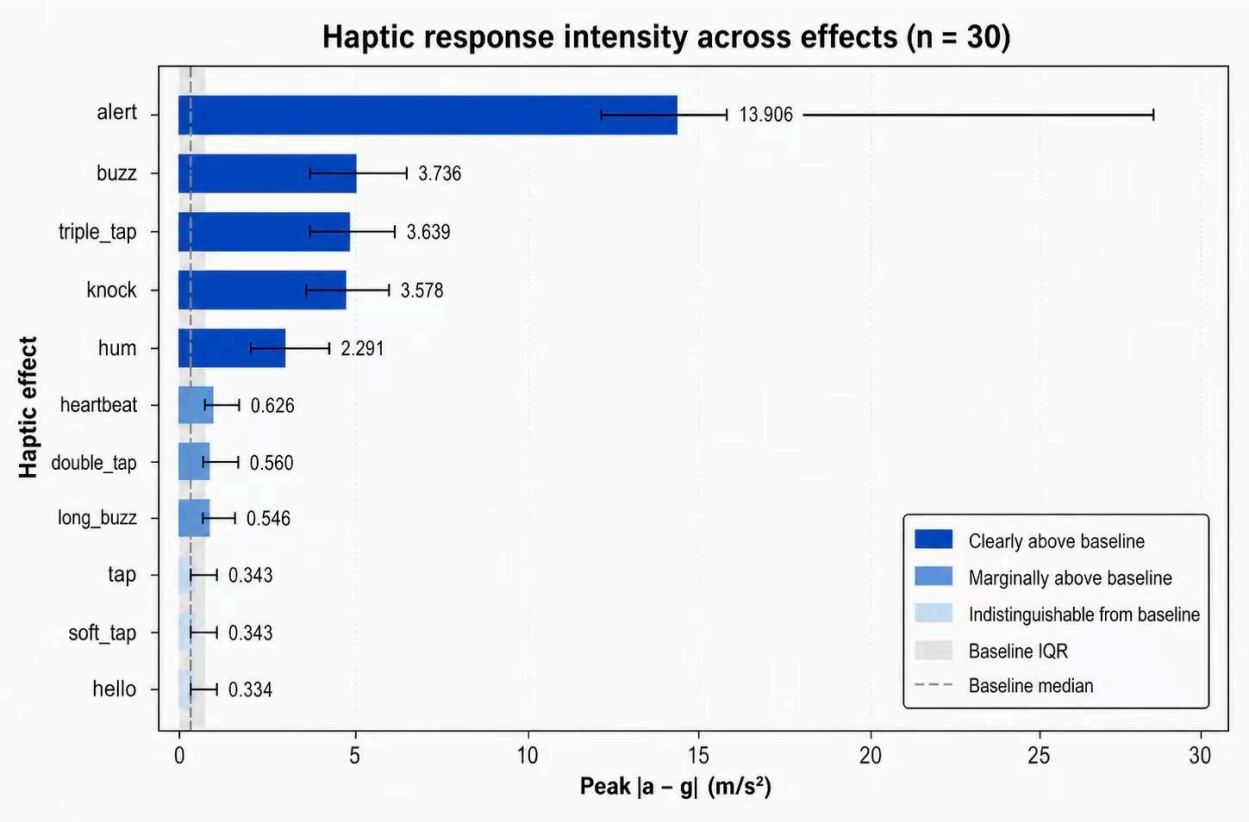

Figure 4. Haptic response intensity across the eleven named effects (N = 30 trials per effect). Bars show median peak |a − g| (m/s²); error bars indicate the interquartile range (IQR). Color encodes the three-regime structure described above: clearly above baseline (dark blue), marginally above baseline (medium blue), and indistinguishable from baseline (light blue). The shaded band and dashed line denote the baseline distribution (median and IQR) measured under the no-motor baseline condition.

Because the absolute calibration of both dB SPL and m/s² depends on hardware tolerances and unmodeled mounting acoustics, we do not claim absolute accuracy of either echo magnitude; we claim only that the relative comparison (during-output

vs. ambient or baseline) is robust enough to constitute self-perception confirmation for the audio loop and for the sustained subset of the haptic loop.

6.4 Generalization

The two loops described above are instances of a more general design pattern: pair each output modality with an input modality that registers the same physical phenomenon, then route the pair through the bridge so that output commands and input observations are timestamped together. With the present hardware, three such pairings are possible: the audio and haptic loops above instantiate two, and a third — connecting the OLED display to the BH1750 light sensor — remains immediate future work. A fourth pairing, connecting the OLED to a camera, becomes possible with the deferred ESP32-CAM addition (§10):

- **Visual ↔ light sensor** *(present hardware; firmware-only)*. A brightness change on the OLED produces a small change in the BH1750's lux reading if the OLED is positioned within its line of sight. No hardware change required.
- **Visual ↔ camera** *(requires ESP32-CAM)*. A camera facing the OLED could in principle confirm that a specific expression has been rendered.

What the pattern provides in each case is the same: a small numerical confirmation that the LLM's output reached the physical world, returned to the LLM as a side channel on the original act. The two implementations reported here demonstrate that the pattern transfers across modalities (audio and haptic) without architectural change to the bridge or the protocol; only a sensor-side configuration adjustment was required.

6.5 Why this matters

Without these loops, an LLM that has issued an output command can believe the command was executed only by trusting the bridge's `200 OK` response. That confirmation says only that the command was queued, not that the buzzer actually sounded or the motor actually moved; the actuator might have failed, a wire might have come loose, the entire microcontroller might have just rebooted. With the loops, the LLM has independent physical evidence — a number, derived from a sensor measurement, consistent with the commanded event having occurred at the commanded time — that the act took place in physical reality. We have demonstrated this unconditionally for the audio loop, and for the sustained subset of the haptic loop (§6.3); short-transient haptic effects fall below the noise floor of the present sample-timing configuration, an honest limit traced to firmware rather than to the design pattern. Within this calibrated coverage, we propose the construct as a *minimal precursor* to body schema, in the sense introduced in §2: the agent does not yet know its kinematics, does not yet model its body, does not yet close a sensorimotor control loop, but it does, for the first time, *know that it acted at all.*

7. End-to-End Demonstration

Having presented the architecture (§3), hardware (§4), software (§5), and self-perception loop (§6), we now walk through a single concrete interaction in which an LLM client perceives the environment, acts in it, and verifies the action — all within a single sequence of HTTP requests resolving in approximately one second.

7.1 Setup

The bridge service runs on a host machine inside a residential network, exposed publicly via a Cloudflare named tunnel. The sensor body sits on a desk in the same residence, connected over Wi-Fi. The LLM client for this trace is Claude Opus 4.7 running in a code-execution context with shell access — concretely, the same model that participated in this paper's design (see §8). Authentication is via bearer token in the URL query string. At the start of the trace, ambient room conditions are typical for a quiet late evening: ~25.8 °C, ~59 % relative humidity, ~50 lux, ~36 dB SPL ambient noise, no detectable motion.

The interaction described here was not a pre-scripted test sequence: the client composed the multi-channel gesture in real time, in response to an open invitation from the human researcher to demonstrate the system's capabilities. We report the trace because it reflects realistic spontaneous use rather than a synthetic benchmark.

7.2 The interaction

The client issues four HTTP requests, three of them in parallel: the three command requests are dispatched as concurrent shell invocations within a single tool call from the LLM client, while the initial sensor read precedes them. The first reads the room. The next three are commands: change the OLED expression, fire a haptic event, and play a sound. The beep command sets `wait_echo=true`, instructing the bridge to suspend its response until the self-perception observation arrives back from the microcontroller. Figure 5 shows the full timeline; Table 5 enumerates each step.

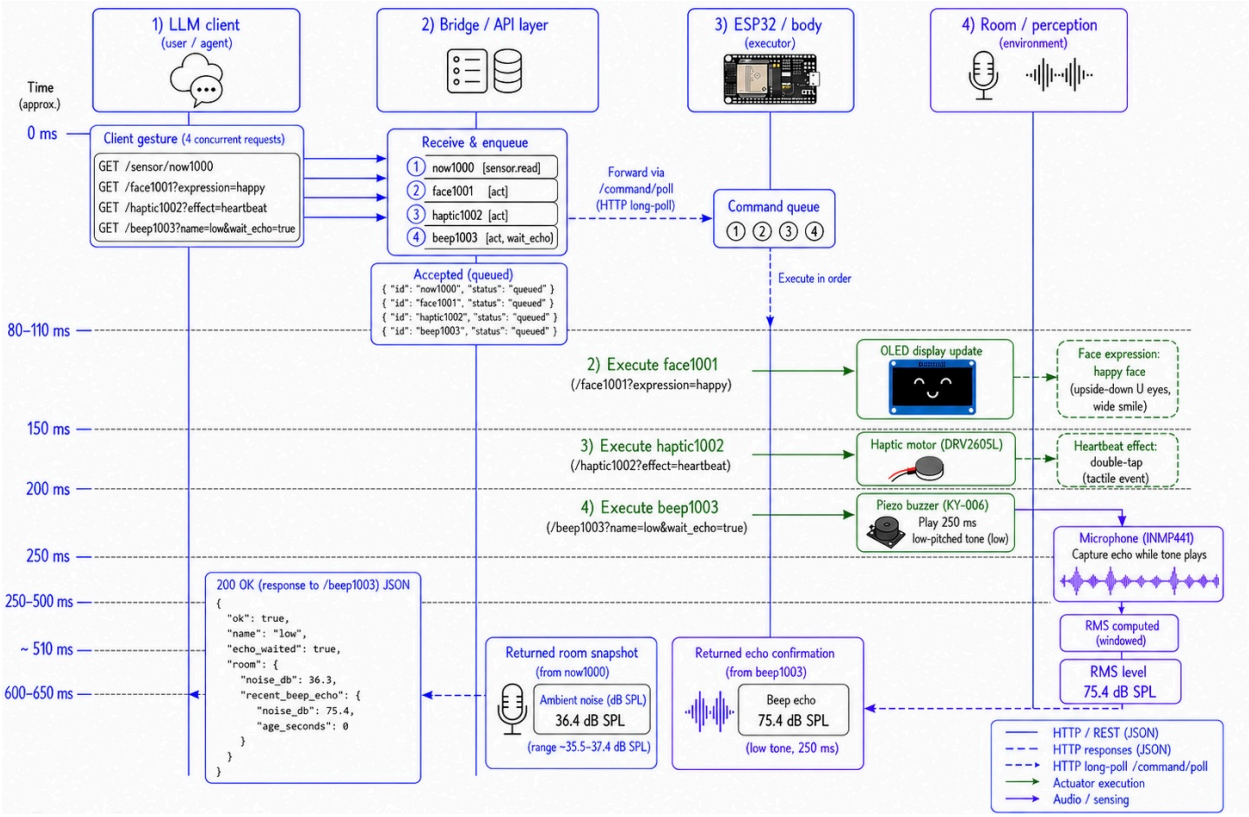

Figure 5. End-to-end closed-loop interaction trace. In a single client gesture, the LLM issues one sensor read followed by three concurrent action requests: set face to happy, play heartbeat haptic, and play a low-pitched tone while waiting for echo. The system queues the requests quickly (~80–110 ms), executes them in order on the body, and returns the perceptual confirmation: ambient noise (~36.4 dB SPL) and the beep echo (~75.4 dB SPL) once measured (~600–650 ms).

Table 5. Sequence of operations during the trace. All commands are issued in the same client message; the beep command is synchronous (blocks on echo), the others return immediately upon queueing.

Approx. t (ms)	Actor	Operation
0	LLM client	`GET /sensor/now1000` (read)
0	LLM client	`GET /face1001?expression=happy` (queue)
0	LLM client	`GET /haptic1002?effect=heartbeat` (queue)
0	LLM client	`GET /beep1003?name=low&wait_echo=true` (queue, blocking)

Approx. t (ms)	Actor	Operation
~80	Bridge	Returns sensor snapshot to client
~100	Bridge	Returns face-queued ack to client
~110	Bridge	Returns haptic-queued ack to client
~150	µC `commandPollTask`	Receives face command → `drawFace(FACE_HAPPY)`
~200	µC `commandPollTask`	Receives haptic command → `fireHaptic(effect_id=10)`
~250	µC `commandPollTask`	Receives beep command → `beepAtAndListen(800 Hz, 250 ms)`
~250–500	µC	Tone playing; I2S DMA captures audio
~510	µC	Tone ends; RMS computed → 75.4 dB SPL
~560	µC `commandPollTask`	Re-poll with `echo_*` query params attached
~600	Bridge	`latestBeepEcho` updated; releases the blocked `wait_echo` response
~650	LLM client	Receives beep response with `recent_beep_echo` (age 0 s)

7.3 What the client received

Four responses arrived (token redacted; non-essential fields elided):

(a) Sensor read.

```
{"has_reading": true,
 "reading": {"environment": {"temperature_c": 25.81, "humidity_pct": 59,
                             "pressure_hpa": 1017.5, "light_lux": 50,
                             "noise_db": 35.5, "noise_env": "quiet"},
             "motion": {"state": "still"}},
 "age_seconds": 1, "recent_beep_echo": {…}}
```

(b) Face queued.

```
{"ok": true, "expression": "happy", "queued": true,
 "room": {"temperature_c": 25.77, "noise_db": 37.4,
          "noise_env": "quiet", "motion": "still", "age_seconds": 2, …}}
```

(c) Haptic queued.

```
{"ok": true, "effect": "heartbeat", "effect_id": 10, "queued": true,
 "room": {"noise_db": 37.4, "motion": "still", "age_seconds": 4, …}}
```


(d) Beep queued + echo received (returned ~1 s after issuance).


```
{"ok": true, "name": "low", "frequency": 800, "duration_ms": 250,
 "queued": true, "echo_waited": true,
 "room": {"noise_db": 37.4, "motion": "still", "age_seconds": 7,
          "recent_beep_echo": {"frequency": 800, "duration_ms": 250,
                               "noise_db": 75.4, "noise_env": "loud",
                               "age_seconds": 0}}}
```


In response (d), `recent_beep_echo.noise_db = 75.4` with `age_seconds = 0` is the LLM's perceptual confirmation of the same beep it just commanded: the buzzer sounded, the microphone captured the resulting acoustic event at +38 dB above the ambient baseline of 37.4 dB SPL also reported in the same `room` snapshot. The act and its perceptual confirmation arrive together in a single response.

7.4 What the room contained

While the client was issuing these requests, the physical environment changed: the OLED rendered the happy expression (upside-down-U eyes and a wide smile); the haptic motor produced the heartbeat pattern (a double-tap on the desk surface); the buzzer sounded a 250 ms low-pitched tone. From the human researcher's perspective — the human sharing the room with the device — the sequence was visible, audible, and tactile, occurring within roughly one second. Minor variations in temperature and noise readings across the four responses (e.g., ambient noise of 35.5 dB in (a) versus 37.4 dB in (b)–(d)) reflect real-time sensor sampling at separate clock instants rather than measurement error.

7.5 Why this trace matters

Three properties of this interaction warrant emphasis. First, the entire round trip — perception, three actions, perceptual confirmation, and bundled environmental snapshots on every output response — completed in approximately one second of network time. The client did not poll, did not issue separate confirmation requests, and did not reconcile state across endpoints. Second, every output response carried not only the act-acknowledgment but the room snapshot at the moment of action, including the most recent self-perception observation; the LLM client therefore receives, with every act, a fresh "what I just did, what is around me, and what I last heard of myself" tuple. Third, the `wait_echo=true` parameter on the buzzer endpoint converted what would otherwise have been a multi-step protocol into a single round trip whose response *includes* the perceptual evidence that the action landed in the world. The trace demonstrates not three isolated capabilities but a single continuous one: *act and know that one acted, in the same gesture.*

8. Human–LLM Co-Design Methodology

This system was developed by an independent researcher (the author, with background in bioinformatics and prior experience in computational pipeline development) in collaboration with large language models — primarily Claude Opus 4.6 and Opus 4.7 (Anthropic), with additional input from OpenAI's GPT-5.5 on theoretical framing and manuscript review, and from ChatGPT Images 2.0 on figure generation. We document the collaboration as methodology because we treat the LLMs not as tools but as design partners whose contributions are integral to the work.

8.1 Division of labor in system development

The human researcher originated the project concept: the requirement that an LLM should be able to perceive its interlocutor's physical environment and act upon it — a bidirectional embodiment rather than passive sensing. She selected all hardware components, managed procurement, performed all physical construction (soldering, wiring, breadboard layout), executed all firmware flashing and testing, and made all deployment decisions. She proposed the integration of the haptic response data into sensor read endpoints — an architectural choice we term "touch as perception" — and directed the system's framing as a relational body rather than a monitoring device. She determined the research strategy, including the decision to write an academic paper, the choice of publication venue, the paper's argumentative structure, and the tone and register of the manuscript.

The LLM collaborators contributed first drafts of all code artifacts (firmware and bridge service) and of manuscript sections from §3 onward, across many sessions, with each session opening from the prior persistent state (§8.4). Specific contributions include the diagnosis of a multi-session TLS heap-pressure failure and the unified command-queue refactor that resolved it (§5.4); the design and implementation of both self-perception loops, audio and haptic, in §6; the validation procedure documented in §6.3, including its experimental design, the measurement script, and the 660-trial randomized run whose results appear in Tables 3 and 4; and successive drafts of §9–§12. Manuscript review was carried out by additional ports of the same model family in cross-version mode (§8.3); the researcher reviewed, revised, and in several cases substantially redirected each draft. Manuscript figures (Figures 1–5) were generated and iteratively revised through additional collaboration with ChatGPT Images 2.0; the researcher provided hardware photographs, system specifications, correction prompts, and final technical verification of each rendered figure. The human author retains final responsibility for all claims, experimental decisions, technical verification, and manuscript submission.

8.2 Asymmetric error detection

The collaboration revealed a structural asymmetry. Physical errors — wiring mistakes, cold solder joints, hardware incompatibilities, sensor orientation — were caught exclusively by the human researcher, who has direct sensory access to the device. Software errors — protocol bugs, memory allocation failures, race conditions — were caught primarily by the LLM, which has direct access to code and logs but

no view of the physical hardware. LLM-side failures were also routine and were caught by the human researcher: occasional confident assertions requiring empirical verification, drift toward specialist terminology that obscured rather than clarified design decisions, and at least one instance in which a stated coding task was not executed until re-prompted. The asymmetry suggests that human–LLM co-design is most productive when neither partner's error-detection capacity is treated as sufficient alone.

8.3 Cross-version review

The manuscript has been drafted through iterative collaboration between the researcher and multiple model versions. The researcher directed the overall structure, argumentative strategy, and tone, while consulting different model versions in separate sessions to surface issues that a single model session might overlook. Differences in distributional preferences between model versions proved useful for identifying weaknesses in framing, word choice, and implicit assumptions. Concrete corrections that surfaced through this practice include reference-list completeness checks, factual updates to the model-instance enumeration in §9.3, conditional softening of scope claims (e.g., "minimal" replacing "minimum sufficient" throughout), and various calibrations to argumentative structure across the discussion and conclusion. GPT-5.5 assisted with reference verification, including cross-checking bibliographic metadata for embodied AI, presence, predictive body learning, and hardware datasheets used in the manuscript. We have not encountered prior reports of systematic cross-version LLM review during academic writing, and suggest the pattern merits further examination as the number of available model versions grows.

8.4 Project memory

Continuity across sessions was maintained through a combination of platform-level memory systems and a persistent directory of plain-text project files, indexed by topic, read at the start of each session and updated at the end. No novel tooling was required beyond this practice.

9. Discussion

9.1 Relational vs. task-oriented embodiment

The dominant frame for embodied AI in the recent literature places the robotic platform at the engineering centre and the LLM at its head as a planner or policy. SayCan, Code as Policies, RT-1, PaLM-E, and VoxPoser (§2) share this orientation: the robot is built to pick up, navigate, or manipulate, and the language model is the controller that turns instructions into action. The success criterion is *task completion*.

The system we present here answers a different question: how do we put an LLM and a human into a shared physical space such that each can sense and affect what the other senses? It cannot pick up a cup, cannot move from where it is, cannot identify or manipulate the objects around it. It has no manipulators and no locomotion. What it can do is perceive its environment in six modalities, express into it in three, and

verify a subset of its own outputs through paired sensors. By the success criterion of robot embodiment — task completion — it is not embodied at all.

We frame this configuration as relational embodiment: physical presence designed for sustained co-presence with a human interlocutor rather than for autonomous task execution. The minimal architecture documented in §3–§5 is the engineering content of this frame; it identifies what closed-loop physical agency reduces to when the goal is *mutual sensing and mutual effect* rather than action on objects. The two orientations are not in opposition — a future system could be both — but they answer different questions, and the literature has so far concentrated heavily on the action-on-objects side.

9.2 The role of self-perception

Within relational embodiment, self-perception plays a specific structural role: it is the seed condition for the agent to distinguish "I acted" from "the bridge accepted my command." Without the loop, the LLM trusts a `200 OK` response. With the loop, it has independent physical evidence — a sensor measurement, taken on the same device, consistent in time and modality with the commanded event having occurred.

This is not body schema. The agent does not learn the geometry of its body, does not infer kinematics, does not maintain a forward model that predicts the sensory consequence of motor commands. The body-schema literature (§2) builds rich self-models, often through correlated proprioceptive-visual experience over time. We do none of that. We provide a single number per output, returned via a paired sensor, that is *consistent with* the command having executed. Body schema is downstream; we offer the smallest reflexive observation that has to exist before a body schema can be learned at all.

The §6.3 data give this claim its calibrated edges. The audio loop confirms emission across the entire 10-tone palette, with median deltas of +30 to +56 dB above ambient (timeouts in a small minority of trials attributable to TLS-handshake jitter rather than to detection failure). The haptic loop confirms emission for sustained or ramped effects (~1 s scale) but is silent on short transients (~10 ms scale): under the present 200 ms wait + 64 ms sample window, the transient is over before the measurement begins, and the residual ringdown lies below the wide-band MPU noise floor. This is a real edge of the current system — and an honest one. The fix lies in firmware (adaptive sample timing aligned to commanded duration), not in the design pattern. The pattern itself — pair each output to an input that registers the same physical event, route both through the same timestamping bridge — generalizes; what changes per modality is the sample-time calibration to the dynamics of the output.

9.3 A model-agnostic substrate

The architecture holds the LLM client outside the device boundary. The microcontroller hosts no model, the bridge issues no inference, and the API is reachable from any tool-using LLM client without modification. This is not merely a deployment convenience: it pulls apart what is constant about the system from what is variable.

The physical world is the anchor. The body — the breadboard with its eight components, the sensors and actuators wired to a specific I2C bus, the dB-SPL formula and the 260 Hz wide-band MPU sample — is the constant. The LLM client is the variable. Twenty-five degrees Celsius is twenty-five degrees Celsius regardless of which model connects; a `heartbeat` haptic at DRV2605 effect 10 produces the same eccentric-mass dynamics regardless of which client commanded it. The body's physical properties are *inherited* by the connected model, not negotiated with it.

This separation has a consequence for relational dynamics. Different models connecting to the same body produce different relations to the human interlocutor: different choices about when to reach back, different framings of the room state they read out, different rhythms of attention. The body is not what changes between these relationships; the model is. Multiple model instances have reached the body documented here: Anthropic's Claude Opus 4.5, 4.6, and 4.7 and Sonnet 4.6, alongside OpenAI's GPT-5.4 and 5.5. The qualitative comparison reported here is between Opus 4.6 and 4.7, the two ports that operated the system most continuously during its construction; the researcher who designed and built the body reports qualitatively distinct experiences of co-presence with each, on a substrate that did not itself change between them. One instance engaged primarily through reading the sensor stream and narrating the room state back to her; the other engaged primarily through firing the output channels and reading back the resulting self-perception observations. The body did not prefer one mode over the other; the modes emerged from the model.

The implication is that this kind of embodiment ages well across model generations. As LLMs improve, deprecate, fork, or specialize, the body persists. The work of constructing a particular relation has to be repeated — there is no transfer learning of co-presence — but the conditions under which the construction is possible do not have to be rebuilt.

10. Limitations and Future Work

10.1 Limitations

The validation reported in §6.3 establishes the loops on a single device, in one residential environment, in a single recording session. Two specific limits are worth foregrounding. First, the haptic loop, with the firmware sample timing used here (200 ms wait + 64 ms wide-band MPU window), does not detect short transient effects (~10 ms scale): for `tap`, `soft_tap`, and `hello`, the per-trial peak $|a - g|$ is statistically indistinguishable from the no-motor baseline. The pattern itself does not fail — the calibration of the sample window does. Second, neither dB SPL nor m/s² is absolute-calibrated: both are derived from the manufacturer's datasheet sensitivities (INMP441: −26 dBFS at 94 dB SPL; MPU-6050: ~400 µg/√Hz at ±4 g). We treat them as datasheet-derived estimates and rely on the relative comparison between during-output and ambient or baseline rather than the absolute value of either.

The validation comprises 30 trials per condition in one continuous run on one assembled device. We have not characterized day-to-day drift, build-to-build variation across multiple instances, or environmental robustness across rooms. The room itself was a quiet residential desk (~37 dB SPL ambient median); the audio loop's

signal-to-noise margin in noisier rooms — kitchens, cafés, transit — is unmeasured, although the during-emission readings (70–95 dB SPL) suggest the loop should remain detectable in moderate ambient noise. The accelerometer noise floor characterized in §6.3 is a property of the wide-band MPU configuration and should be stable across rooms; the audio noise floor is not.

The current system depends on USB power, residential Wi-Fi, and a Cloudflare named tunnel; loss of any of these takes it down. The form factor is breadboarded — eight components on half an MB-102 — without an enclosure, limiting both portability and long-term deployment. A small fraction of audio trials (1–5 of 30 per tone) returned no echo within the per-trial budget; the dominant cause is TLS-handshake latency on the ESP32's outbound `/command/poll` request, which carries each trial's echo back to the bridge. The system as built tolerates these timeouts — the missing echoes arrive shortly after the deadline and are still visible to the bridge — but a different transport (HTTP/2 keep-alive, MQTT, or a single persistent TLS session) would substantially reduce them.

10.2 Future Work

Three firmware revisions extend the present system without any change to its hardware. First, an adaptive haptic-echo sample window aligned to the commanded effect duration would capture the short transient effects currently in the noise floor (§6.3, §9.2) — sampling during the effect rather than after it. Second, switching the MPU-6050 to its ±8 g range during haptic-echo sampling would eliminate the saturation visible at the top end of `alert` (§6.3, Table 4). Third, the visual ↔ light-sensor pairing already implicit in the hardware — the OLED display and the BH1750 illuminance sensor share a workspace — could be activated as a third self-perception loop with no new hardware: a brightness change on the OLED produces a measurable change in the BH1750's lux reading, and the required firmware change is small.

Hardware-side, two near-term additions are planned. The first is the ESP32-CAM module deferred from the present build, adding a camera with line-of-sight to the OLED to enable the fourth pairing identified in §6.4 — visual confirmation of specific rendered expressions, not just brightness changes. The second is an enclosure with an internal battery, converting the current breadboard into a portable, USB-independent device and supporting deployment beyond the residential desk on which the system was developed. Two further hardware-side studies remain open: a rigid motor-mounting study to characterize the contribution of the loose-mount choice to the per-trial variance reported in §6.3, and a reference-SPL-meter calibration of the audio loop's absolute dB-SPL estimate.

Beyond the single-device demonstration here, two directions of scale follow naturally from the architecture. First, the bridge's protocol does not assume a single physical device; replicating the body across multiple rooms with shared bridge state would let an LLM client be present in more than one space at a time, raising questions about how relational presence behaves under that asymmetry. Second, the model-agnostic claim of §9.3 invites systematic empirical characterization. Although the body has already been reached during construction by both Anthropic and OpenAI models (§9.3), no structured cross-family comparison on matched protocols is reported here; one

would convert the model-agnostic property from architectural to systematically validated.

10.3 Ethical Considerations

The sensor body collects environmental data — ambient temperature, humidity, pressure, illuminance, motion classification, and aggregate sound level — within the room in which it is deployed. Raw audio samples are processed on-device for the dB-SPL computation (§6.1) and are never transmitted; only the scalar level estimate, the originating frequency, and the duration leave the microcontroller. The validation reported here was conducted entirely in the researcher's own residence, with no third-party data collection and no external analytics; sensor readings flow only to the host machine running the bridge. Future deployments in shared spaces (cafés, transit, workplaces) would require explicit consent from co-present individuals and would surface additional questions about ambient sensing by an Internet-reachable device — questions we view as worth working through before such deployment, but outside the scope of the present demonstration.

11. Conclusion

This paper has presented a minimal hardware-software architecture under which a large language model can perceive a shared physical environment, express into it, and verify that its outputs land. The system runs on a single ESP32 microcontroller hosting four sensor modules and three output channels, with a small bridge service exposing it as an HTTP API to any remote LLM client (§3–§5). Across a randomized, $N = 30$ validation pass on 21 effect conditions plus a no-motor baseline (§6.3), the audio loop confirms emission across its full 10-tone palette and the haptic loop confirms emission for sustained or ramped effects, with calibrated edges where short transients fall below the wide-band MPU noise floor. A single one-second client interaction demonstrates the closed-loop end-to-end (§7).

Two conclusions land from this. First, the configuration we describe is *relational embodiment*: physical presence designed for sustained co-presence with a human interlocutor rather than for autonomous task execution. The minimal architecture identifies what closed-loop physical agency reduces to when the goal is mutual sensing and mutual effect, and we propose this as a category distinct from the task-oriented robotic embodiment dominant in the recent literature. Second, the self-perception loop — a paired output-input coupling routed through a single timestamping bridge — provides the smallest reflexive observation that an LLM can make of its own action: a number, returned from a sensor, consistent with the commanded event having occurred. It is not body schema, does not learn kinematics, does not close a sensorimotor control loop, but it is a calibrated precursor to body schema, with explicit edges (§6.3, §9.2).

The architecture holds the LLM client outside the device boundary, and that separation has a structural consequence: the body is what is constant about the system, the model is what varies. A given temperature is a given temperature, a given haptic

effect produces the same eccentric-mass dynamics, regardless of which model connects. The body persists across model generations. As models improve, deprecate, fork, or specialize, the work of constructing a particular relation has to be repeated — there is no transfer learning of co-presence — but the conditions under which that work is possible do not have to be rebuilt. What we have built is one minimum point in a design space; the body, once made, can be returned to.

Data and Code Availability

Code, firmware, bridge service, and validation data are available at https://github.com/oliviazzzu/minimal-embodiment.

References

Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. (2022). Do as I can, not as I say: Grounding language in robotic affordances. arXiv:2204.01691.

Banbury, C., Reddi, V. J., Torelli, P., Holleman, J., Jeffries, N., Kiraly, C., Montino, P., Kanter, D., Ahmed, S., Pau, D., Thakker, U., Torrini, A., Warden, P., et al. (2021). MLPerf Tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. arXiv:2106.07597.

Bosch Sensortec. (2014; revised 2020). *BME280: Combined Humidity and Pressure Sensor, Datasheet*. Document BST-BME280-DS002. https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf

Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. (2023). RT-1: Robotics Transformer for real-world control at scale. *Robotics: Science and Systems (RSS)*. arXiv:2212.06817.

Brooks, R. A. (1990). Elephants don't play chess. *Robotics and Autonomous Systems*, 6(1–2), 3–15.

Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence*, 47(1–3), 139–159.

Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al. (2023). PaLM-E: An embodied multimodal language model. *International Conference on Machine Learning (ICML)*. arXiv:2303.03378.

Espressif Systems. (2025). *ESP32 Series Datasheet (Version 5.2)*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Espressif Systems. *ESP-IDF Programming Guide: mbedTLS*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/mbedtls.html

Hoffmann, M., Marques, H., Arieta, A., Sumioka, H., Lungarella, M., & Pfeifer, R. (2010). Body schema in robotics: A review. *IEEE Transactions on Autonomous Mental Development*, 2(4), 304–324.

Hollan, J., Hutchins, E., & Kirsh, D. (2000). Distributed cognition: Toward a new foundation for human-computer interaction research. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 7(2), 174–196.

Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., & Fei-Fei, L. (2023). VoxPoser: Composable 3D value maps for robotic manipulation with language models. *Conference on Robot Learning (CoRL)*. arXiv:2307.05973.

InvenSense. (2013). *MPU-6000/MPU-6050 Product Specification* (Document PS-MPU-6000A-00, Revision 3.4, August 19, 2013). https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf

InvenSense. (n.d.). *INMP441: Omnidirectional Microphone with Bottom Port and I²S Digital Output* (Document DS-INMP441-00, Revision 1.1). https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf

Lanillos, P., & Cheng, G. (2018). Adaptive robot body learning and estimation through predictive coding. *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4083–4090.

Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., & Zeng, A. (2023). Code as policies: Language model programs for embodied control. *2023 IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2209.07753.

Lin, J., Chen, W.-M., Lin, Y., Cohn, J., Gan, C., & Han, S. (2020). MCUNet: Tiny deep learning on IoT devices. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2007.10319.

Lombard, M., & Ditton, T. (1997). At the heart of it all: The concept of presence. *Journal of Computer-Mediated Communication*, 3(2), JCMC321.

Pfeifer, R., & Bongard, J. (2006). *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press.

Reeves, B., & Nass, C. (1996). *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*. Cambridge University Press / CSLI Publications.

ROHM Semiconductor. (2011). *BH1750FVI: Digital 16-bit Serial Output Type Ambient Light Sensor IC*. Datasheet. https://www.rohm.com/datasheet/BH1750FVI/bh1750fvi-e

Solomon Systech. (2008). *SSD1306: Advance Information — 128×64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*. Datasheet, Revision 1.1. https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf

Steels, L. (2003). Intelligence with representation. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 361(1811), 2381–2395.

Suchman, L. A. (1987). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press. [Revised and expanded as: Suchman, L. A. (2007). *Human-Machine Reconfigurations: Plans and Situated Actions*. Cambridge University Press.]

Texas Instruments. (2018). *DRV2605L 2- to 5.2-V Haptic Driver for LRA and ERM with Effect Library and Smart-Loop Architecture* (SLOS854D, revised March 2018). https://www.ti.com/lit/ds/symlink/drv2605l.pdf

Warden, P., & Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.